1. Student Performance API Development

A college requires a RESTful service to manage student scores. Design endpoints to retrieve a student's marks using their registration ID and to modify their scores. Include validation for incorrect or missing data.

Expected Outcome:

Functional GET and PUT/PATCH endpoints

Responses structured in JSON with appropriate HTTP status codes

Input validation and error responses umplemented

ANSWER

3. Create Spring Boot Project (Eclipse)

Step 1:

Open Eclipse →
File → New → Spring Starter Project

Step 2:

Fill details:

Name: student-api
Type: Maven
Packaging: Jar
Java: 17
Group: com.college
Artifact: studentapi

Step 3: Add Dependencies

Select:

- Spring Web
- Spring Boot DevTools
- Validation (IMPORTANT)

Click **Finish**

📁 4. Project Structure

After creation, arrange like this:

```
studentapi
└── src/main/java
    └── com.college.studentapi
        ├── StudentApiApplication.java
        ├── controller
        │   └── StudentController.java
        ├── model
        │   └── Student.java
        ├── service
        │   └── StudentService.java
        └── exception
            ├── ResourceNotFoundException.java
            └── GlobalExceptionHandler.java
```

🧱 5. Create Packages

Right-click com.college.studentapi →
👉 New → Package → create:

- controller
- model
- service
- exception

👨‍🎓 6. Student Model (Paste this)

📄 model/Student.java

```
package com.college.studentapi.model;

import jakarta.validation.constraints.NotNull;

public class Student {

    @NotNull(message = "Registration ID is required")
    private String regId;
```

```
    @NotNull(message = "Marks cannot be null")
    private Integer marks;

    public Student() {}

    public Student(String regId, Integer marks) {
        this.regId = regId;
        this.marks = marks;
    }

    public String getRegId() {
        return regId;
    }

    public void setRegId(String regId) {
        this.regId = regId;
    }

    public Integer getMarks() {
        return marks;
    }

    public void setMarks(Integer marks) {
        this.marks = marks;
    }
}
```

⚙️ 7. Service Layer (Logic)

📄 service/StudentService.java

```
package com.college.studentapi.service;

import java.util.HashMap;
import java.util.Map;

import org.springframework.stereotype.Service;

import com.college.studentapi.model.Student;
import com.college.studentapi.exception.ResourceNotFoundException;
```

```
@Service
public class StudentService {

  private Map<String, Student> db = new HashMap<>();

  public StudentService() {
    db.put("101", new Student("101", 85));
    db.put("102", new Student("102", 90));
  }

  public Student getStudent(String regId) {
    Student s = db.get(regId);
    if (s == null) {
      throw new ResourceNotFoundException("Student not found");
    }
    return s;
  }

  public Student updateStudent(String regId, Student student) {
    if (!db.containsKey(regId)) {
      throw new ResourceNotFoundException("Student not found");
    }
    db.put(regId, student);
    return student;
  }
}
```

🌐 **8. Controller (Endpoints)**

📄 controller/StudentController.java

```
package com.college.studentapi.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.validation.annotation.Validated;
import org.springframework.web.bind.annotation.*;

import com.college.studentapi.model.Student;
import com.college.studentapi.service.StudentService;
```

```
import jakarta.validation.Valid;

@RestController
@RequestMapping("/students")
@Validated
public class StudentController {

  @Autowired
  private StudentService service;

  // GET endpoint
  @GetMapping("/{regId}")
  public ResponseEntity<Student> getStudent(@PathVariable String regId) {
    return ResponseEntity.ok(service.getStudent(regId));
  }

  // PUT endpoint
  @PutMapping("/{regId}")
  public ResponseEntity<Student> updateStudent(
      @PathVariable String regId,
      @Valid @RequestBody Student student) {

    return ResponseEntity.ok(service.updateStudent(regId, student));
  }

  // PATCH endpoint
  @PatchMapping("/{regId}")
  public ResponseEntity<Student> patchStudent(
      @PathVariable String regId,
      @RequestBody Student student) {

    Student existing = service.getStudent(regId);

    if (student.getMarks() != null) {
      existing.setMarks(student.getMarks());
    }

    return ResponseEntity.ok(service.updateStudent(regId, existing));
  }
}
```

❗ 9. Custom Exception

📄 exception/ResourceNotFoundException.java

```
package com.college.studentapi.exception;

public class ResourceNotFoundException extends RuntimeException {
  public ResourceNotFoundException(String msg) {
    super(msg);
  }
}
```

🛑 10. Global Exception Handler

📄 exception/GlobalExceptionHandler.java

```
package com.college.studentapi.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

  @ExceptionHandler(ResourceNotFoundException.class)
  public ResponseEntity<Map<String, String>>
handleNotFound(ResourceNotFoundException ex) {
    Map<String, String> error = new HashMap<>();
    error.put("error", ex.getMessage());
    return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
  }

  @ExceptionHandler(Exception.class)
  public ResponseEntity<Map<String, String>> handleGeneral(Exception ex) {
    Map<String, String> error = new HashMap<>();
    error.put("error", "Invalid request");
```

```
        return new ResponseEntity<>(error, HttpStatus.BAD_REQUEST);
    }
}
```

▶️ 11. Run the Project

Step:

1. Right-click project
2. Run As → **Spring Boot App**

Output:

Tomcat started on port 8080

🧪 12. Test API (Use Browser/Postman)

✅ GET Student

GET http://localhost:8080/students/101

Response:

```
{
  "regId": "101",
  "marks": 85
}
```

✅ PUT Update Student

PUT http://localhost:8080/students/101

Body:

```
{
  "regId": "101",
  "marks": 95
}
```

✅ PATCH Update Marks Only

PATCH http://localhost:8080/students/101

Body:

```
{
  "marks": 99
}
```

❌ **Error Case**

GET /students/999

Response:

```
{
  "error": "Student not found"
}
```

---

2. Multi-Format REST Response Handling

Upgrade an existing REST API so that it can deliver responses on either JSON or XML format depending on client preferences specified in request headers.

Expected Outcome

Single endpoint serving multiple formats (JSON/XML)

Proper use of content negotiation mechanism

Correct configuration for serialization deserialization

ANSWER

STEP 1: Create Project in Eclipse

1. Open **Eclipse**
2. Go to:
   File → New → Spring Starter Project
3. Enter:

Name: multiformat-api
Group: com.college

Artifact: multiformat
Packaging: Jar
Java: 17

4. Click **Next**
5. Select dependencies:

- ✅ Spring Web
- ✅ Validation

6. Click **Finish**

📦 STEP 2: Add XML Dependency

Open pom.xml → paste inside <dependencies>:

```
<dependency>
  <groupId>com.fasterxml.jackson.dataformat</groupId>
  <artifactId>jackson-dataformat-xml</artifactId>
</dependency>
```

👉 Save the file

📁 STEP 3: Create Packages

Inside:
src/main/java/com.college.multiformat

👉 Right click → New → Package → create:

- model
- service
- controller
- config

📄 STEP 4: Model Class

📄 model/Student.java

```
package com.college.multiformat.model;
```

```
import com.fasterxml.jackson.dataformat.xml.annotation.JacksonXmlRootElement;

@JacksonXmlRootElement(localName = "student")
public class Student {

    private String regId;
    private int marks;

    public Student() {}

    public Student(String regId, int marks) {
        this.regId = regId;
        this.marks = marks;
    }

    public String getRegId() {
        return regId;
    }

    public void setRegId(String regId) {
        this.regId = regId;
    }

    public int getMarks() {
        return marks;
    }

    public void setMarks(int marks) {
        this.marks = marks;
    }
}
```

📄 **STEP 5: Service Class**

📄 service/StudentService.java

```
package com.college.multiformat.service;

import java.util.HashMap;
import java.util.Map;
```

```
import org.springframework.stereotype.Service;

import com.college.multiformat.model.Student;

@Service
public class StudentService {

   private Map<String, Student> db = new HashMap<>();

   public StudentService() {
      db.put("101", new Student("101", 85));
      db.put("102", new Student("102", 90));
   }

   public Student getStudent(String regId) {
      return db.get(regId);
   }
}
```

📄 **STEP 6: Controller (MAIN LOGIC)**

📄 controller/StudentController.java

```
package com.college.multiformat.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import com.college.multiformat.model.Student;
import com.college.multiformat.service.StudentService;

@RestController
@RequestMapping("/students")
public class StudentController {

   @Autowired
   private StudentService service;

   @GetMapping(
      value = "/{regId}",
      produces = { "application/json", "application/xml" }
```

```
    )
    public Student getStudent(@PathVariable String regId) {
        return service.getStudent(regId);
    }
}
```

📄 **STEP 7: Config Class (IMPORTANT for Browser)**

📄 config/WebConfig.java

```
package com.college.multiformat.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.http.MediaType;
import
org.springframework.web.servlet.config.annotation.ContentNegotiationConfigurer;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void configureContentNegotiation(ContentNegotiationConfigurer configurer) {
        configurer
            .favorParameter(true) // enable ?format=
            .parameterName("format")
            .ignoreAcceptHeader(true)
            .defaultContentType(MediaType.APPLICATION_JSON)
            .mediaType("json", MediaType.APPLICATION_JSON)
            .mediaType("xml", MediaType.APPLICATION_XML);
    }
}
```

📄 **STEP 8: application.properties**

📍 Location:
src/main/resources/application.properties

Paste:

spring.mvc.contentnegotiation.favor-parameter=true
spring.mvc.contentnegotiation.parameter-name=format

▶️ STEP 9: Run the Project

1. Right click project
2. Click **Run As → Spring Boot App**

Console:

Tomcat started on port 8080

🌐 STEP 10: Test in Browser (IMPORTANT)

✅ JSON

Open browser:

http://localhost:8080/students/101?format=json

✓ Output:

```
{
 "regId": "101",
 "marks": 85
}
```

✅ XML

Open browser:

http://localhost:8080/students/101?format=xml

✓ Output:

```
<student>
  <regId>101</regId>
  <marks>85</marks>
</student>
```

---

3. AJAX-Based Item Management Enhancement

A web application currently supports adding and removing records dynamically using AJAX .Extend this system to allow users to edit/update existing records without refreshing the page

Expected Outcome:

Update functionality implemented using AJAX calls

Changes reflected dynamically on the user interface

No full-page reload required

ANSWER

STEP 1: Create Project in Eclipse

1. Open **Eclipse**
2. Go to:
   File → New → Spring Starter Project
3. Enter:

Name: ajax-item-api
Group: com.college
Artifact: ajaxitem
Packaging: Jar
Java: 17

4. Click **Next**
5. Select dependency:

- ✅ Spring Web

6. Click **Finish**

📁 **STEP 2: Project Structure**

```
ajaxitem
└── src/main/java/com.college.ajaxitem
    ├── AjaxitemApplication.java
    ├── controller
```

```
│   └── ItemController.java
├── model
│   └── Item.java
└── service
    └── ItemService.java

└── src/main/resources
    ├── static
    │   └── index.html
    └── application.properties
```

📦 STEP 3: Create Packages

Inside com.college.ajaxitem → create:

- model
- service
- controller

📄 STEP 4: Model Class

📄 model/Item.java

```
package com.college.ajaxitem.model;

public class Item {

  private int id;
  private String name;

  public Item() {}

  public Item(int id, String name) {
    this.id = id;
    this.name = name;
  }

  public int getId() {
    return id;
  }
```

```
    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}
```

📄 **STEP 5: Service Class**

📄 service/ItemService.java

```
package com.college.ajaxitem.service;

import java.util.*;

import org.springframework.stereotype.Service;

import com.college.ajaxitem.model.Item;

@Service
public class ItemService {

    private Map<Integer, Item> db = new HashMap<>();
    private int count = 1;

    public List<Item> getAll() {
        return new ArrayList<>(db.values());
    }

    public Item add(Item item) {
        item.setId(count++);
        db.put(item.getId(), item);
        return item;
    }
```

```
  public Item update(int id, Item item) {
    if (db.containsKey(id)) {
      item.setId(id);
      db.put(id, item);
      return item;
    }
    return null;
  }

  public void delete(int id) {
    db.remove(id);
  }
}
```

📄 **STEP 6: Controller (API)**

📄 controller/ItemController.java

```
package com.college.ajaxitem.controller;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import com.college.ajaxitem.model.Item;
import com.college.ajaxitem.service.ItemService;

@RestController
@RequestMapping("/items")
@CrossOrigin
public class ItemController {

  @Autowired
  private ItemService service;

  @GetMapping
  public List<Item> getAll() {
    return service.getAll();
  }
```

```
    @PostMapping
    public Item add(@RequestBody Item item) {
        return service.add(item);
    }

    @PutMapping("/{id}")
    public Item update(@PathVariable int id, @RequestBody Item item) {
        return service.update(id, item);
    }

    @DeleteMapping("/{id}")
    public void delete(@PathVariable int id) {
        service.delete(id);
    }
}
```

🌐 **STEP 7: Frontend (AJAX) — MAIN PART**

📄 src/main/resources/static/index.html

👉 Right-click static → New → File → index.html

Paste:

```
<!DOCTYPE html>
<html>
<head>
  <title>Item Manager</title>
</head>
<body>

<h2>Item Manager (AJAX)</h2>

<input type="text" id="name" placeholder="Enter item">
<button onclick="addItem()">Add</button>

<ul id="list"></ul>

<script>
const API = "http://localhost:8080/items";
```

```
function loadItems() {
  fetch(API)
  .then(res => res.json())
  .then(data => {
    let list = document.getElementById("list");
    list.innerHTML = "";
    data.forEach(item => {
      list.innerHTML += `
        <li>
          ${item.name}
          <button onclick="editItem(${item.id}, '${item.name}')">Edit</button>
          <button onclick="deleteItem(${item.id})">Delete</button>
        </li>`;
    });
  });
}

function addItem() {
  let name = document.getElementById("name").value;

  fetch(API, {
    method: "POST",
    headers: {"Content-Type": "application/json"},
    body: JSON.stringify({name: name})
  }).then(() => {
    loadItems();
    document.getElementById("name").value = "";
  });
}

function deleteItem(id) {
  fetch(API + "/" + id, { method: "DELETE" })
  .then(() => loadItems());
}

// 🔥 UPDATE USING AJAX (MAIN QUESTION)
function editItem(id, oldName) {
  let newName = prompt("Edit item:", oldName);

  if (newName) {
    fetch(API + "/" + id, {
```

```
        method: "PUT",
        headers: {"Content-Type": "application/json"},
        body: JSON.stringify({name: newName})
      }).then(() => loadItems());
    }
}

loadItems();
</script>

</body>
</html>
```

▶️ STEP 8: Run Project

1. Right-click project
2. Run As → **Spring Boot App**

🌐 STEP 9: Open in Browser

http://localhost:8080

---

4. Resolving Cross-Origin Communication Issues

A web client is unable to interact with a backend service due to Cross-Origin restrictions. Update the backend configuration to allow secure cross-origin requests from trusted domains

Expected Outcome:

Proper CORS policy configuration

Secure handling of allowed origins, methods, and headers

ANSWER

STEP 1: Create Project in Eclipse

1. Open **Eclipse**
2. Click:
   File → New → Spring Starter Project
3. Enter:

Name: cors-api
Group: com.college
Artifact: corsapi
Packaging: Jar
Java: 17

4. Click **Next**
5. Select dependency:

- ✅ Spring Web

6. Click **Finish**

📁 STEP 2: Project Structure (You must create this)

```
corsapi
└── src/main/java/com.college.corsapi
    ├── CorsapiApplication.java
    ├── controller
    │   └── StudentController.java
    └── config
        └── CorsConfig.java
```

📦 STEP 3: Create Packages

Inside com.college.corsapi:

👉 Right click → **New → Package**

Create:

- controller
- config

📄 STEP 4: Create Controller (for API)

👉 Right click controller → New → Class → **StudentController**

Paste:

```
package com.college.corsapi.controller;

import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api")
public class StudentController {

  @GetMapping("/data")
  public String getData() {
    return "CORS Enabled Response";
  }
}
```

🔐 **STEP 5: Create CORS Configuration (MAIN ANSWER)**

👉 Right click config → New → Class → **CorsConfig**

Paste:

```
package com.college.corsapi.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.*;

@Configuration
public class CorsConfig implements WebMvcConfigurer {

  @Override
  public void addCorsMappings(CorsRegistry registry) {

    registry.addMapping("/api/**")
        .allowedOrigins("http://localhost:3000") // trusted domain
        .allowedMethods("GET", "POST", "PUT", "DELETE")
        .allowedHeaders("*")
        .allowCredentials(true);
  }
}
```

▶️ STEP 6: Run the Project

1. Right click project
2. Click → **Run As → Spring Boot App**

Console will show:

Tomcat started on port 8080

🌐 STEP 7: Check Output

Open browser:

http://localhost:8080/api/data

Output:

CORS Enabled Response

---

5. Modernizing a Legacy Web Application

Convert a traditional servlet - based asynchronous web application into a modern front end architecture using frameworks like React or Vue. Ensure seamless communication with backend services

Expected Outcomes

Implementation of a component-driven UI

Integration with backend APIs using fetch or axios

Smooth user experience without page reloads

ANSWER

STEP 0: INSTALL REQUIRED SOFTWARE (Linux Terminal)

Open **Terminal (Ctrl + Alt + T)**

Install Node.js (for React)

```
sudo apt update
sudo apt install nodejs npm -y
node -v
npm -v
```

Install Java (if not installed)

```
sudo apt install openjdk-17-jdk -y
java -version
```

Install Apache Tomcat

```
sudo apt install tomcat10 -y
```

🧱 STEP 1: CREATE BACKEND PROJECT (Eclipse)

👉 Open **Eclipse**

Create Dynamic Web Project

1. File → New → **Dynamic Web Project**
2. Project Name: StudentBackend
3. Target Runtime: **Apache Tomcat**
4. Click Finish

📁 PROJECT STRUCTURE (Backend)

```
StudentBackend/
├── src/
│   └── com.example
│       ├── Student.java
│       └── StudentServlet.java
├── WebContent/
│   └── WEB-INF/
│       └── web.xml
```

📦 STEP 2: CREATE PACKAGE

👉 In Eclipse:

- Right-click **src**

- New → Package → com.example

📜 STEP 3: CREATE MODEL CLASS

👉 Right-click package → New → Class → Student.java

Paste this:

```
package com.example;

public class Student {
  private int id;
  private String name;
  private int marks;

  public Student(int id, String name, int marks) {
    this.id = id;
    this.name = name;
    this.marks = marks;
  }

  public String toJSON() {
    return "{ \"id\": " + id + ", \"name\": \"" + name + "\", \"marks\": " + marks + " }";
  }
}
```

🌐 STEP 4: CREATE SERVLET

👉 New Class → StudentServlet.java

Paste this:

```
package com.example;

import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.*;
import javax.servlet.http.*;

public class StudentServlet extends HttpServlet {

  protected void doGet(HttpServletRequest request, HttpServletResponse response)
```

```
        throws ServletException, IOException {

    response.setContentType("application/json");
    response.setHeader("Access-Control-Allow-Origin", "*"); // CORS

    PrintWriter out = response.getWriter();

    Student s = new Student(1, "Varshsan", 90);

    out.print(s.toJSON());
    out.flush();
  }
}
```

⚙️ STEP 5: CONFIGURE web.xml

👉 Go to: WebContent → WEB-INF → web.xml

Paste:

```
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee" version="3.1">

  <servlet>
    <servlet-name>student</servlet-name>
    <servlet-class>com.example.StudentServlet</servlet-class>
  </servlet>

  <servlet-mapping>
    <servlet-name>student</servlet-name>
    <url-pattern>/student</url-pattern>
  </servlet-mapping>

</web-app>
```

▶️ STEP 6: RUN BACKEND

👉 In Eclipse:

- Right-click project → Run As → **Run on Server**
- Choose Tomcat

👉 Test in browser:

http://localhost:8080/StudentBackend/student

✅ Output:

{ "id": 1, "name": "Varshsan", "marks": 90 }

⚛️ STEP 7: CREATE REACT FRONTEND (Terminal)

👉 Go to Terminal (NOT Eclipse)

```
npx create-react-app student-frontend
cd student-frontend
npm start
```

Browser opens:

http://localhost:3000

📁 FRONTEND STRUCTURE

```
student-frontend/
 ├── src/
 │   ├── App.js
 │   └── Student.js
```

🧩 STEP 8: CREATE COMPONENT

👉 Go to:

student-frontend/src

Create file: Student.js

Paste:

```
import React, { useEffect, useState } from "react";

function Student() {
  const [student, setStudent] = useState({});

  useEffect(() => {
    fetch("http://localhost:8080/StudentBackend/student")
```

```
    .then(res => res.json())
    .then(data => setStudent(data));
  }, []);

  return (
    <div>
      <h2>Student Details</h2>
      <p>ID: {student.id}</p>
      <p>Name: {student.name}</p>
      <p>Marks: {student.marks}</p>
    </div>
  );
}

export default Student;
```

🔗 STEP 9: CONNECT COMPONENT

👉 Open App.js

Replace with:

```
import React from "react";
import Student from "./Student";

function App() {
  return (
    <div>
      <h1>Modern Web App</h1>
      <Student />
    </div>
  );
}

export default App;
```

▶️ STEP 10: RUN FRONTEND

👉 In Terminal:

```
npm start
```

🎯 FINAL OUTPUT

Browser:

http://localhost:3000

Shows:

Modern Web App
Student Details
ID: 1
Name: Varshsan
Marks: 90

💡 IMPORTANT CONCEPTS (for exam)

✅ Component-Based UI

- Student.js is a reusable component

✅ API Integration

- fetch() used to call servlet

✅ No Page Reload

- React updates UI dynamically

✅ CORS Handling

response.setHeader("Access-Control-Allow-Origin", "*");